

Analysis of 450k DNA methylation data with minfi

Kasper D. Hansen

- [Dependencies](#)
- [Overview](#)
- [Other Resources](#)
- [DNA methylation](#)
- [Array Design](#)
- [Data](#)
- [Preprocessing](#)
- [Differential Methylation](#)
- [SessionInfo](#)
- [References](#)

Dependencies

This document has the following dependencies:

```
library(minfi)
library(GEOquery)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("minfi", "GEOquery"))
```

Overview

The Illumina 450k DNA methylation microarray is a cheap and relatively comprehensive array for assaying DNA methylation. It is the platform of choice for large sample profiling of DNA methylation, especially for so-called EWAS (Epigenome-wide association studies). The studies are like GWAS (genome-wide association studies) but instead of associated a phenotype (like disease) with genotype (typically measured on SNP arrays or exome/whole-genome sequencing) they associated phenotype with epigenotype.

Other Resources

- The vignette from the [minfi webpage](#).

DNA methylation

DNA methylation is a chemical modification of DNA. In humans, it occurs at CpG dinucleotides where the 'C' can be methylated or not. The methylation state of a given locus in a single cell is binary (technically tertiary since we have two copies of most chromosomes) but we measure DNA methylation across a population of cells. We observe that some loci has intermediate methylation values (between 0 and 1) and we use the methylation percentage (or Beta-value) to describe this.

The goal of most analyses of DNA methylation is to associate changes in phenotype with changes in methylation in one or more loci.

Array Design

The 450k array has a very unusual design, which to some extent impact analysis. It is really a mixture of a two-color array and two one-color arrays. There are two main types of probes (type I and type II) and the probe design affects the signal distribution of the probe.

The raw data format for the 450k array is known as IDAT. Because the array is measured in two different colors, there are two files for each sample, typically with the extension `_Grn.idat` and `_Red.idat`. Illumina's software suite for analysis of this array is called GenomeStudio. It is not unusual for practitioners to only have access to processed data from GenomeStudio instead of the raw IDAT files, but I and others have shown that there is information in the IDAT files which are beneficial to analysis.

Data

We will access a dataset created with the intention of studying acute mania. Serum samples were obtained from individuals hospitalized with acute mania as well as unaffected controls.

We want to obtain the IDAT files which are available as supplementary data. Far from all 450k datasets on GEO has IDAT files available. First we download the files.

```
library(GEOquery)
getGEOSuppFiles("GSE68777")
untar("GSE68777/GSE68777_RAW.tar", exdir = "GSE68777/idad")
head(list.files("GSE68777/idad", pattern = "idad"))
```

```
## [1] "GSM1681154_5958091019_R03C02_Grn.idat"  
## [2] "GSM1681154_5958091019_R03C02_Grn.idat.gz"  
## [3] "GSM1681154_5958091019_R03C02_Red.idat"  
## [4] "GSM1681154_5958091019_R03C02_Red.idat.gz"  
## [5] "GSM1681155_5935446005_R05C01_Grn.idat"  
## [6] "GSM1681155_5935446005_R05C01_Grn.idat.gz"
```

Currently *minfi* does not support reading compressed IDAT files. This is clearly a needed functionality and (as the maintainer of this package) I will address this. But for now we will need to decompress the files.

```
idatFiles <- list.files("GSE68777/idat", pattern = "idat.gz$", full = TR  
  sapply(idatFiles, gunzip, overwrite = TRUE)
```



```
## GSE68777/iddat/GSM1681154_5958091019_R03C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681154_5958091019_R03C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681155_5935446005_R05C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681155_5935446005_R05C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681156_5958091020_R01C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681156_5958091020_R01C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681157_5958091020_R03C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681157_5958091020_R03C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681158_5935403032_R05C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681158_5935403032_R05C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681159_5958091019_R04C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681159_5958091019_R04C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681160_5935446005_R06C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681160_5935446005_R06C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681161_5958091020_R02C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681161_5958091020_R02C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681162_5958091020_R04C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681162_5958091020_R04C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681163_5935403032_R06C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681163_5935403032_R06C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681164_5935446005_R01C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681164_5935446005_R01C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681165_5958091020_R03C01_Grn.idat.gz
## 8091452
```

GSE68777/izat/GSM1681165_5958091020_R03C01_Red.idat.gz
8091452
GSE68777/izat/GSM1681166_5958091020_R05C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681166_5958091020_R05C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681167_5935403032_R01C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681167_5935403032_R01C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681168_5958091019_R06C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681168_5958091019_R06C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681169_5935446005_R02C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681169_5935446005_R02C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681170_5958091020_R04C01_Grn.idat.gz
8091452
GSE68777/izat/GSM1681170_5958091020_R04C01_Red.idat.gz
8091452
GSE68777/izat/GSM1681171_5958091020_R06C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681171_5958091020_R06C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681172_5935403032_R02C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681172_5935403032_R02C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681173_5958091019_R05C01_Grn.idat.gz
8091452
GSE68777/izat/GSM1681173_5958091019_R05C01_Red.idat.gz
8091452
GSE68777/izat/GSM1681174_5935446005_R01C01_Grn.idat.gz
8091452
GSE68777/izat/GSM1681174_5935446005_R01C01_Red.idat.gz
8091452
GSE68777/izat/GSM1681175_5935446005_R03C02_Grn.idat.gz
8091452
GSE68777/izat/GSM1681175_5935446005_R03C02_Red.idat.gz
8091452
GSE68777/izat/GSM1681176_5958091020_R05C01_Grn.idat.gz
8091452
GSE68777/izat/GSM1681176_5958091020_R05C01_Red.idat.gz
8091452
GSE68777/izat/GSM1681177_5935403032_R01C01_Grn.idat.gz

```
## 8091452
## GSE68777/iddat/GSM1681177_5935403032_R01C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681178_5935403032_R03C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681178_5935403032_R03C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681179_5958091019_R06C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681179_5958091019_R06C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681180_5935446005_R02C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681180_5935446005_R02C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681181_5935446005_R04C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681181_5935446005_R04C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681182_5958091020_R06C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681182_5958091020_R06C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681183_5935403032_R02C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681183_5935403032_R02C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681184_5958091019_R01C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681184_5958091019_R01C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681185_5935446005_R03C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681185_5935446005_R03C01_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681186_5935446005_R05C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681186_5935446005_R05C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681187_5958091020_R01C02_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681187_5958091020_R01C02_Red.idat.gz
## 8091452
## GSE68777/iddat/GSM1681188_5935403032_R03C01_Grn.idat.gz
## 8091452
## GSE68777/iddat/GSM1681188_5935403032_R03C01_Red.idat.gz
## 8091452
```

```
## GSE68777/idad/GSM1681189_5958091019_R02C02_Grn.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681189_5958091019_R02C02_Red.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681190_5935446005_R04C01_Grn.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681190_5935446005_R04C01_Red.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681191_5935446005_R06C02_Grn.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681191_5935446005_R06C02_Red.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681192_5958091020_R02C02_Grn.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681192_5958091020_R02C02_Red.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681193_5935403032_R04C01_Grn.idat.gz
##                                     8091452
## GSE68777/idad/GSM1681193_5935403032_R04C01_Red.idat.gz
##                                     8091452
```

Now we read the IDAT files using `read.450k.exp()` which (in this case) reads all the IDAT files in a directory.

```
rgSet <- read.450k.exp("GSE68777/idad")
rgSet
```

```
## RGChannelSet (storageMode: lockedEnvironment)
## assayData: 622399 features, 40 samples
##   element names: Green, Red
## An object of class 'AnnotatedDataFrame': none
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
```

```
pData(rgSet)
```

```
## data frame with 0 columns and 40 rows
```

```
head(sampleNames(rgSet))
```

```
## [1] "GSM1681154_5958091019_R03C02" "GSM1681155_5935446005_R05C01"  
## [3] "GSM1681156_5958091020_R01C01" "GSM1681157_5958091020_R03C02"  
## [5] "GSM1681158_5935403032_R05C01" "GSM1681159_5958091019_R04C02"
```

Now we have the data, but note that we have no pheno data. And the filenames are very unhelpful here. These names consists of a GEO identifier (the GSM part) followed by a standard IDAT naming convention with a 10 digit number which is an array identifier followed by an identifier of the form R01C01. This is because each array actually allows for the hybridization of 12 samples in a 6x2 arrangement. The 5958091020_R01C0 means row 1 and column 1 on chip 5958091020. This is all good, but does not help us understand which samples are cases and which are controls.

We now get the standard GEO representation to get the phenotype data stored in GEO. Most of the columns in this phenotype data are irrelevant (contains data such as the address of the person who submitted the data); we keep the useful ones. Then we clean it.

```
geoMat <- getGEO("GSE68777")  
pD.all <- pData(geoMat[[1]])  
pD <- pD.all[, c("title", "geo_accession", "characteristics_ch1.1", "cha  
head(pD)
```

```
##           title geo_accession characteristics_ch1.1  
## GSM1681154 5958091019_R03C02   GSM1681154      diagnosis: Mania  
## GSM1681155 5935446005_R05C01   GSM1681155      diagnosis: Mania  
## GSM1681156 5958091020_R01C01   GSM1681156      diagnosis: Ctr  
## GSM1681157 5958091020_R03C02   GSM1681157      diagnosis: Ctr  
## GSM1681158 5935403032_R05C01   GSM1681158      diagnosis: Mania  
## GSM1681159 5958091019_R04C02   GSM1681159      diagnosis: Mania  
##           characteristics_ch1.2  
## GSM1681154           Sex: Female  
## GSM1681155           Sex: Female  
## GSM1681156           Sex: Male  
## GSM1681157           Sex: Female  
## GSM1681158           Sex: Female  
## GSM1681159           Sex: Male
```

```
names(pD)[c(3,4)] <- c("group", "sex")
pD$group <- sub("^diagnosis: ", "", pD$group)
pD$sex <- sub("^Sex: ", "", pD$sex)
```

We now need to merge this pheno data into the methylation data. To do so, we need a common sample identifier and we make sure we re-order the phenotype data in the same order as the methylation data. Finally we put the phenotype data inside the methylation data.

```
sampleNames(rgSet) <- sub(".*_5", "5", sampleNames(rgSet))
rownames(pD) <- pD$title
pD <- pD[sampleNames(rgSet),]
pData(rgSet) <- pD
rgSet
```

```
## RGChannelSet (storageMode: lockedEnvironment)
## assayData: 622399 features, 40 samples
##   element names: Green, Red
## An object of class 'AnnotatedDataFrame'
##   sampleNames: 5958091019_R03C02 5935446005_R05C01 ...
##       5935403032_R04C01 (40 total)
##   varLabels: title geo_accession group sex
##   varMetadata: labelDescription
## Annotation
##   array: IlluminaHumanMethylation450k
##   annotation: ilmn12.hg19
```

Preprocessing

The `rgSet` object is a class called `RGChannelSet` which represents two color data with a green and a red channel, very similar to an `ExpressionSet`.

The first step is usually to preprocess the data, using a number of functions including

- `preprocessRaw()` : do nothing.
- `preprocessIllumina()` : use Illumina's standard processing choices.
- `preprocessQuantile()` : use a version of quantile normalization adapted to methylation arrays.
- `preprocessNoob()` : use the NOOB background correction method.
- `preprocessSWAN()` : use the SWAN method.
- `preprocessFunnorm()` : use functional normalization.

These functions output different types of objects.

The class hierarchy in minfi is as follows: data can be stored in an Methylation and Unmethylation channel or in a percent methylation (called Beta) channel. For the first case we have the class `MethylSet`, for the second case we have the class `RatioSet`. When you have methylation / unmethylation values you can still compute Beta values on the fly. You convert from a `MethylSet` to a `RatioSet` with `ratioConvert()`.

In addition to these two classes, we have `GenomicMethylSet` and `GenomicRatioSet`. The `Genomic` indicates that the data has been associated with genomic coordinates using the `mapToGenome()` function.

The starting point for most analyses ought to be a `GenomicRatioSet` class. If your preprocessing method of choice does not get you there, use `ratioConvert()` and `mapToGenome()` to go the last steps.

Let us run `preprocessQuantile()` which arrives at a `GenomicRatioSet`:

```
grSet <- preprocessQuantile(rgSet)
```

```
## [preprocessQuantile] Mapping to genome.  
## [preprocessQuantile] Fixing outliers.  
## [preprocessQuantile] Quantile normalizing.
```

```
grSet
```

```
## class: GenomicRatioSet  
## dim: 485512 40  
## exptData(0):  
## assays(2): M CN  
## rownames(485512): cg13869341 cg14008030 ... cg08265308 cg14273923  
## rowRanges metadata column names(0):  
## colnames(40): 5958091019_R03C02 5935446005_R05C01 ...  
##      5958091020_R02C02 5935403032_R04C01  
## colData names(5): title geo_accession group sex predictedSex  
## Annotation  
##   array: IlluminaHumanMethylation450k  
##   annotation: ilmn12.hg19  
## Preprocessing  
##   Method: Raw (no normalization or bg correction)  
##   minfi version: 1.14.0  
##   Manifest version: 0.4.0
```

This is like a SummarizedExperiment; we can get the location of the CpGs by

```
granges(grSet)
```

```
## GRanges object with 485512 ranges and 0 metadata columns:
##           seqnames           ranges strand
##           <Rle>             <IRanges> <Rle>
## cg13869341 chr1 [15865, 15865]      *
## cg14008030 chr1 [18827, 18827]      *
## cg12045430 chr1 [29407, 29407]      *
## cg20826792 chr1 [29425, 29425]      *
## cg00381604 chr1 [29435, 29435]      *
##           ...             ...      ...
## cg17939569 chrY [27009430, 27009430]  *
## cg13365400 chrY [27210334, 27210334]  *
## cg21106100 chrY [28555536, 28555536]  *
## cg08265308 chrY [28555550, 28555550]  *
## cg14273923 chrY [28555912, 28555912]  *
## -----
## seqinfo: 24 sequences from hg19 genome; no seqlengths
```

The usual methylation measure is called “Beta” values; equal to percent methylation and defined as Meth divided by Meth + Unmeth.

```
getBeta(grSet)[1:3,1:3]
```

```
##           5958091019_R03C02 5935446005_R05C01 5958091020_R01C01
## cg13869341      0.7485333      0.7696497      0.7322275
## cg14008030      0.5300410      0.5893653      0.6044354
## cg12045430      0.0912596      0.1007678      0.1057603
```

CpGs forms clusters known as “CpG Islands”. Areas close to CpG Islands are known as CpG Shores, followed by CpG Shelves and finally CpG Open Sea probes. An easy way to get at this is to use

```
head(getIslandStatus(grSet))
```

```
## [1] "OpenSea" "OpenSea" "Island"  "Island"  "Island"  "OpenSea"
```

Differential Methylation

Once the data has been normalized, one possibility is to identify differentially methylated CpGs by using *limma* on the Beta values.

Another possibility is to look for clusters of CpGs all changing in the same direction. One method for doing this is through the `bumphunter()` function which interfaces to the *bumphunter* package.

SessionInfo

```

## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4      parallel    methods     stats       graphics    grDevices    utils
## [8] datasets    base
##
## other attached packages:
## [1] IlluminaHumanMethylation450kanno.ilmn12.hg19_0.2.1
## [2] IlluminaHumanMethylation450kmanifest_0.4.0
## [3] GEOquery_2.34.0
## [4] minfi_1.14.0
## [5] bumpHunter_1.8.0
## [6] locfit_1.5-9.1
## [7] iterators_1.0.7
## [8] foreach_1.4.2
## [9] Biostrings_2.36.4
## [10] xVector_0.8.0
## [11] GenomicRanges_1.20.6
## [12] GenomeInfoDb_1.4.2
## [13] IRanges_2.2.7
## [14] S4Vectors_0.6.5
## [15] lattice_0.20-33
## [16] Biobase_2.28.0
## [17] BiocGenerics_0.14.0
## [18] BiocStyle_1.6.0
## [19] rmarkdown_0.8
##
## loaded via a namespace (and not attached):
## [1] mclust_5.0.2          base64_1.1
## [3] Rcpp_0.12.0           Rsamtools_1.20.4
## [5] digest_0.6.8          plyr_1.8.3
## [7] futile.options_1.0.0  RSQLite_1.0.0
## [9] evaluate_0.7.2        zlibbioc_1.14.0
## [11] GenomicFeatures_1.20.3 annotate_1.46.1
## [13] preprocessCore_1.30.0 splines_3.2.1
## [15] BiocParallel_1.2.20   stringr_1.0.0
## [17] RCurl_1.95-4.7        biomaRt_2.24.0
## [19] rtracklayer_1.28.10   multtest_2.24.0
## [21] pkgmaker_0.22         htmltools_0.2.6
## [23] quadprog_1.5-5        codetools_0.2-14
## [25] matrixStats_0.14.2    XML_3.98-1.3

```

## [27] reshape_0.8.5	GenomicAlignments_1.4.1
## [29] MASS_7.3-43	bitops_1.0-6
## [31] grid_3.2.1	nlme_3.1-121
## [33] xtable_1.7-4	registry_0.3
## [35] DBI_0.3.1	magrittr_1.5
## [37] formatR_1.2	stringi_0.5-5
## [39] genefilter_1.50.0	doRNG_1.6
## [41] limma_3.24.15	futile.logger_1.4.1
## [43] nor1mix_1.2-1	lambda.r_1.1.7
## [45] RColorBrewer_1.1-2	siggenes_1.42.0
## [47] tools_3.2.1	illuminaio_0.10.0
## [49] rngtools_1.2.4	survival_2.38-3
## [51] yaml_2.1.13	AnnotationDbi_1.30.1
## [53] heannlot_1.2	knitr_1.11

References
